

Imports and the Python Standard Library

Day 5 • Block 37 • 30 minutes teaching + 30 minutes exercises



Today: modules, imports, namespaces, and standard-library tools.

Key shift

Professional programmers do not start from zero. They know how to reuse reliable code.

What Students Should Be Able to Do

Imports are the bridge between your code and reusable capabilities.

1 Explain what a Python module is

2 Use import module syntax

3 Use from module import name syntax

4 Explain why namespaces reduce confusion

5 Use standard-library modules such as statistics and datetime

6 Avoid reinventing functionality that Python already provides

Why Imports Matter

You can reuse code instead of rebuilding every tool yourself.

```
# Could write your own average function...
values = [10, 20, 30]

total = 0
for value in values:
    total += value

average = total / len(values)
```

Reinventing a common tool

```
# Or use a reliable standard module
import statistics

values = [10, 20, 30]

average = statistics.mean(values)
```

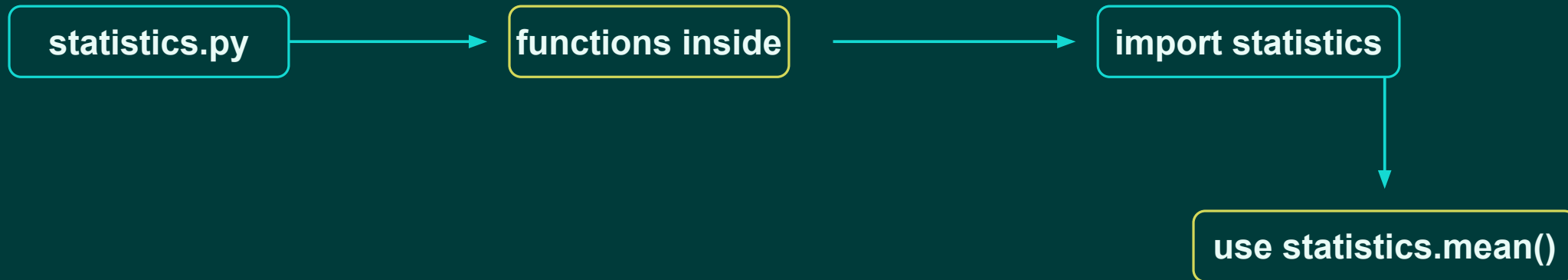
Reusing tested behavior

Mental model

Imports let your program borrow capabilities from another Python file.

A Module Is a Python File With Reusable Code

Some modules are yours. Some come with Python. Some are installed separately.



```
# A module can contain functions,  
# classes, constants, and other code.
```

```
import statistics
```

```
average = statistics.mean(values)
```

Pattern 1: import module

The module name stays attached to the function call.

```
import statistics

values = [10, 20, 30]

average = statistics.mean(values)
median = statistics.median(values)
```

statistics.mean

Read as: use mean from the statistics module.

Beginner-friendly habit

This form is explicit. Readers can see where the function came from.

Pattern 2: from module import name

Import only the specific function you plan to use.

```
from statistics import mean
```

```
values = [10, 20, 30]
```

```
average = mean(values)
```

```
from datetime import date
```

```
today = date.today()
```

```
print(today)
```

Tradeoff: shorter calls, but less obvious where the name came from if overused.

Both import styles are common. The goal is readable code, not shortest code.

Aliases Give a Module a Shorter Name

Use aliases when they improve readability, not just to save typing.

```
import statistics as stats

values = [10, 20, 30]

average = stats.mean(values)
median = stats.median(values)
```

stats = statistics

Aliases can be useful when a module has a common abbreviation.

Avoid confusing abbreviations
like `import statistics as s`

Namespaces Help Prevent Naming Conflicts

The module name acts like a label on its contents.

```
import statistics  
  
mean = "average value"  
  
result = statistics.mean(values)  
  
print(mean)  
print(result)
```

statistics.mean
means the mean function inside
statistics

mean
can be your own variable name

Rule of thumb

Use module prefixes when they make the source of a function clearer.

The Standard Library Comes With Python

Many useful tools are available without installing anything extra.

pathlib

work with paths

csv

read and write CSV files

json

read and write JSON files

statistics

mean, median, and more

datetime

dates and times

math

mathematical functions

random

random choices and numbers

collections

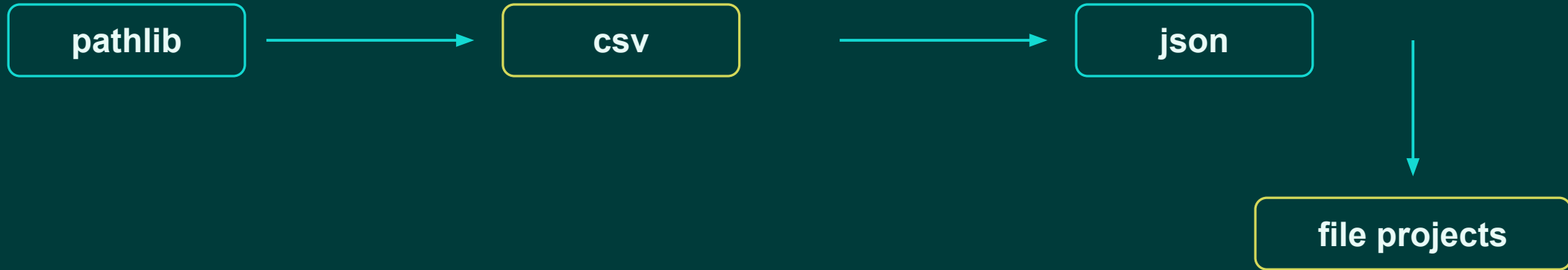
special container types

Big idea

Before building a tool, ask whether Python already provides a reliable one.

You Have Already Used Standard-Library Modules

Day 5 morning used three important examples.



```
from pathlib import Path
import csv
import json
```

These imports solved path, CSV, and JSON problems without third-party packages.

Example: statistics

Use tested functions for common summary calculations.

```
import statistics

record_counts = [12000, 8500, 5000, 17000]

average_count = statistics.mean(record_counts)
median_count = statistics.median(record_counts)

print(average_count)
print(median_count)
```

mean()

typical average

median()

middle value

Teaching note

Students do not need data science yet. They just need to see reuse.

Example: datetime

Use dates without manually building date strings.

```
from datetime import date

today = date.today()

summary = {
    "date_processed": str(today),
    "records_processed": 100,
}

print(summary)
```

date.today()

asks the system for the current date

str(today)

turns the date into text for output

Example: math and random

The standard library includes many small utilities.

```
import math

area = math.pi * radius ** 2

rounded_up = math.ceil(area)
```

math: numeric helpers and constants

```
import random

sample_id = random.choice(record_ids)

print(sample_id)
```

random: sampling and random values

Caution

Randomness is useful for examples and sampling, but it should not be used for security decisions in this course.

Choosing an Import Style

Prefer the form that makes the code easiest to read.

```
import module
```

clear source

```
statistics.mean(values)
```

```
from module import name
```

shorter calls

```
mean(values)
```

```
import module as alias
```

common abbreviation

```
stats.mean(values)
```

```
from module import *
```

usually avoid

where did this name come from?

Avoid wildcard imports because they make code harder to trace and debug.

Imports Usually Go at the Top

A predictable structure helps readers scan a file quickly.

```
import csv
import json
import statistics
from datetime import date
from pathlib import Path

def calculate_summary(records):
    ...

def main():
    ...
```

1. Imports

2. Function definitions

3. Main program flow

Habit

Make dependencies visible before the program logic begins.

Instructor Demo: Add Summary Statistics

Start with records loaded from files, then reuse standard modules.

```
import statistics
from datetime import date

records = [
    {"record_count": 12000, "error_count":
0},
    {"record_count": 8500, "error_count": 2},
    {"record_count": 5000, "error_count": 0},
]

record_counts = []
error_counts = []
```

```
for record in records:

    record_counts.append(record["record_count"]
)

    error_counts.append(record["error_count"])

summary = {
    "date_processed": str(date.today()),
    "average_record_count":
statistics.mean(record_counts),
    "median_error_count":
statistics.median(error_counts),
}
```

Teaching Checkpoint

Students should be able to explain each part before moving to exercises.

1. What does `import statistics` make available?
2. Why does `statistics.mean()` include a dot?
3. What changes when using `from statistics import mean`?
4. Why might `from module import *` be confusing?
5. Which imports used today are standard library modules?

Instructor move

Ask students to read code aloud using “from the module...” language.

Exercise 1: Calculate Average Record Count

Use `statistics.mean()` instead of writing your own average loop.

```
import statistics

record_counts = [12000, 8500, 5000, 17000]

average_count = statistics.mean(record_counts)

print(average_count)
```

Task

Change the list and observe the new average.

Timer

6 minutes

Exercise 2: Calculate Median Processing Time

Use another function from the same module.

```
import statistics

processing_times = [22, 18, 45, 30, 28]

median_time = statistics.median(
    processing_times
)

print(median_time)
```

Task

Add one more time value.
Compare mean and median.

Timer

6 minutes

Exercise 3: Add Today's Date to a Summary

Use `datetime.date` to add useful metadata.

```
from datetime import date

today = date.today()

summary = {
    "date_processed": str(today),
    "records_processed": 20,
    "records_valid": 18,
    "records_rejected": 2,
}

print(summary)
```

Task

Add an `output_file` field and print the full summary.

Timer

6 minutes

Exercise 4: Identify What Each Module Solves

Match the module to the problem it helped solve.

`pathlib`

Create and inspect file paths

`csv`

Read and write tabular records

`json`

Read and write nested structured data

`statistics`

Calculate summary statistics

`datetime`

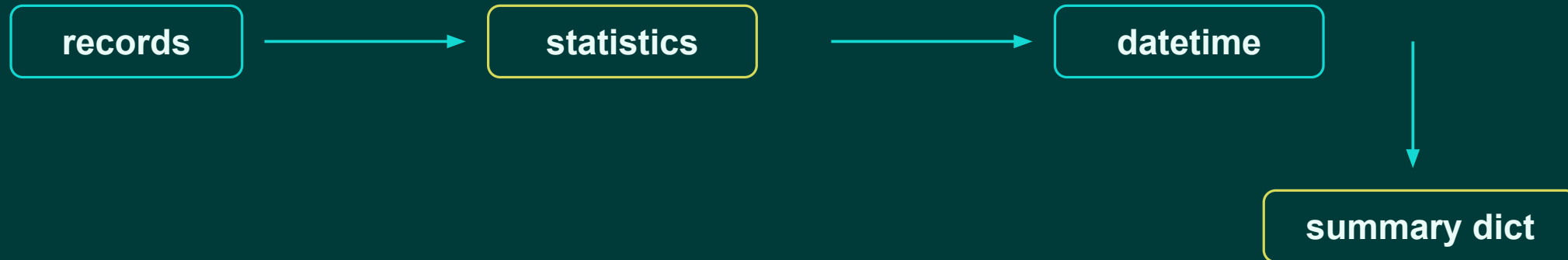
Work with dates and times

Task

For each module, write one sentence: “We used ____ so that ____.”

Applied Challenge: Improve the Morning Summary

Use standard-library modules to make output more useful.



Required fields

`date_processed • average_record_count • median_error_count • input_file •
output_file`

Applied Challenge: Starter Code

Students add imports, calculations, and summary fields.

```
records = [  
    {"record_count": 12000, "error_count": 0},  
    {"record_count": 8500, "error_count": 2},  
    {"record_count": 5000, "error_count": 0},  
]  
  
input_file = "data/raw/equipment.csv"  
output_file = "data/output/summary.json"  
  
summary = {  
    "records_processed": len(records),  
}
```

Tasks

1. Import statistics
2. Import date
3. Build count lists
4. Add average and median
5. Add today's date

Timer

12 minutes

Common Import Errors

Read the error. It usually says what Python could not find.

```
import statistic

# ModuleNotFoundError:
# No module named 'statistic'
```

```
import statistics

average = statistic.mean(values)

# NameError: name 'statistic' is not
defined
```

Fixing process: check spelling → check import style → check the name used at call site.

Many import bugs are mismatches between what was imported and what was called.

Block 37 Takeaways

Imports let programs grow by reusing existing code.

Module a Python file containing reusable code

import module keeps the module name visible

from import brings a specific name into your file

Alias shorter name when it improves readability

Namespace helps readers and Python know where names come from

Standard library useful modules included with Python

Next create your own modules and split projects across files